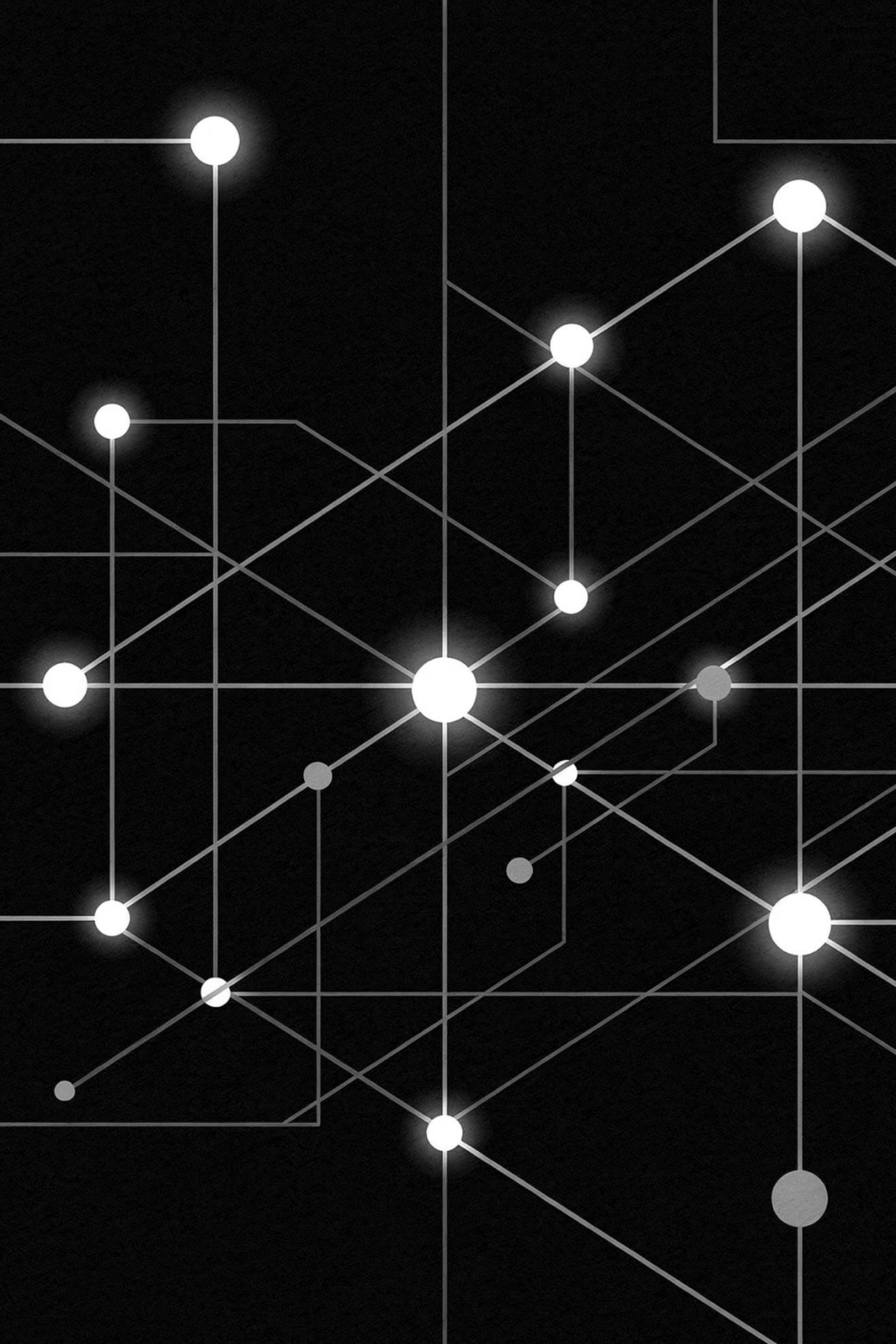




UML Diagrams: State, Object, Package, Component & Deployment

A deep dive into five powerful perspectives that let you explain any system — from idea to production.

Five Angles on Any System

Until now we've focused on structure and interactions — class diagrams, sequence diagrams. Today we go one level deeper, examining a system from five distinct perspectives.

01

How objects change over time

02

What exists at a specific moment

03

How the system is organized

04

How it is built

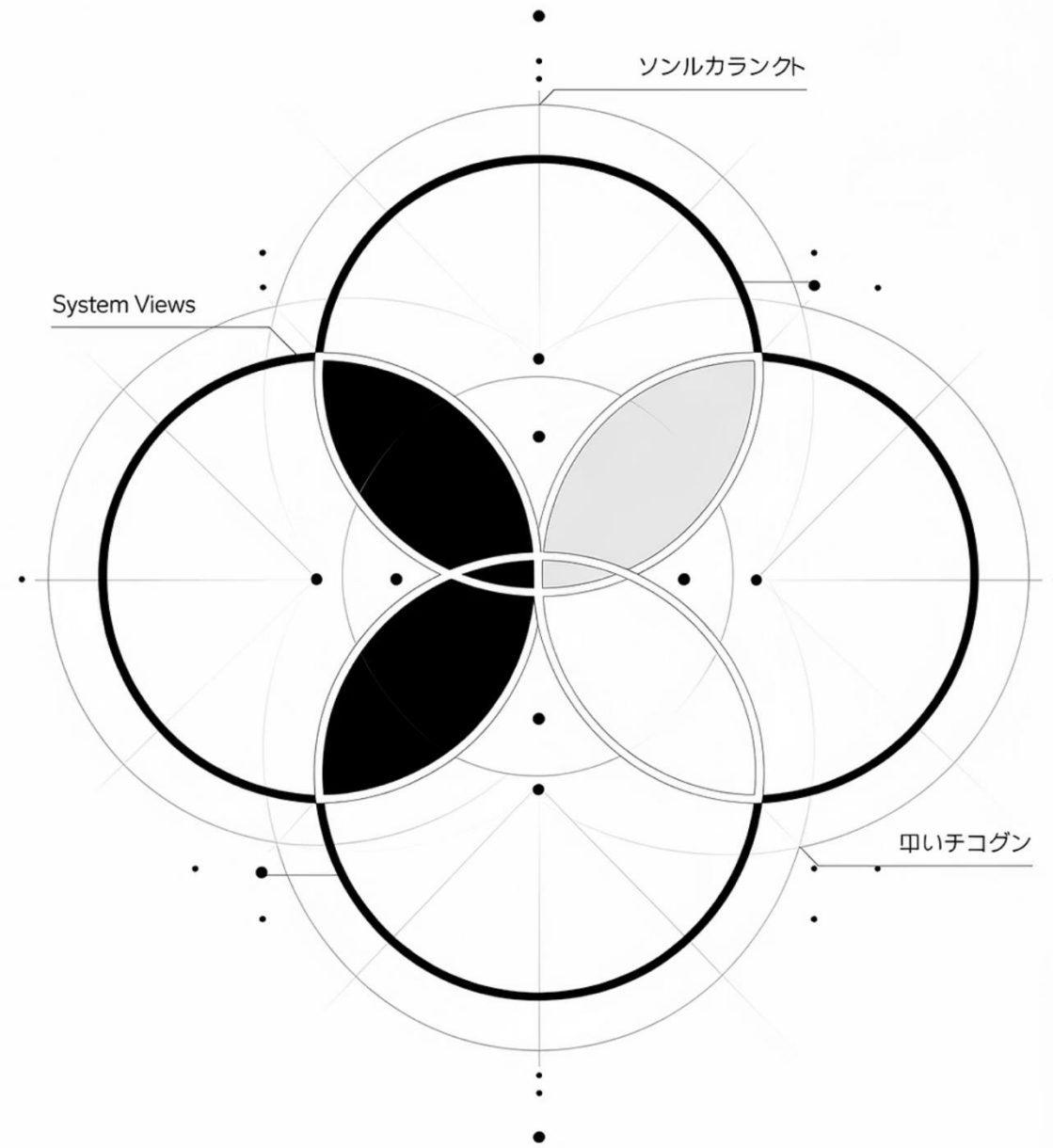
05

Where it actually runs

Understand these five and you can walk into any system and explain it from idea to production.

The Big Picture

Each diagram type answers a fundamentally different question about a system. Together, they form a complete vocabulary for software architecture — from the lifecycle of a single object all the way to the physical machines running in production.



State Diagrams

LIFECYCLE

State diagrams describe the life of a **single object over time**. This is not about interactions between objects — it is about how *one* object evolves through different situations.

Key Idea

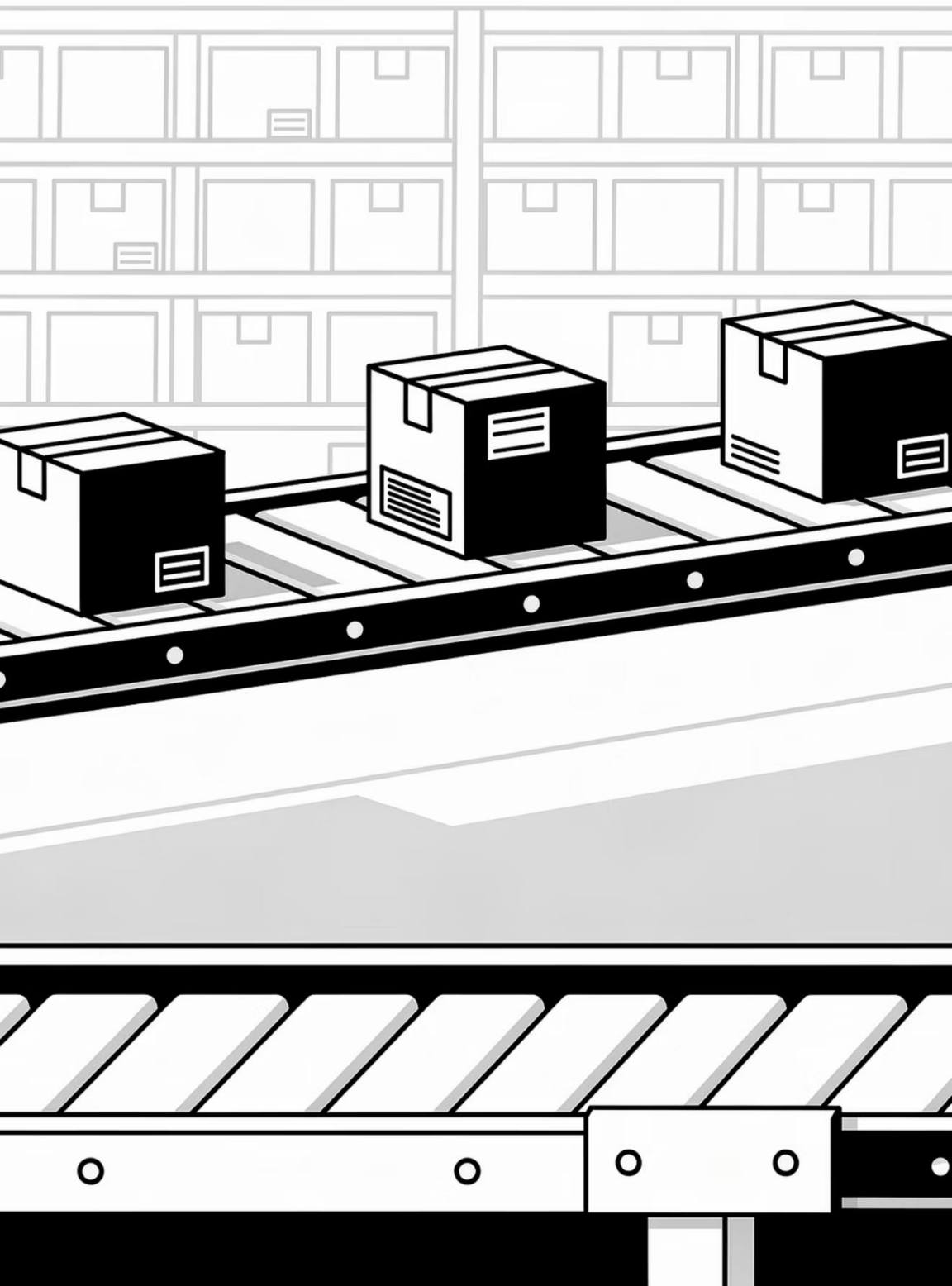
State diagram = lifecycle. This is about evolution, not communication.

States

Each box represents a situation the system can be in at a given moment.

Transitions

Arrows between states, triggered by events that cause the object to evolve.



State Diagram: The Order Lifecycle

Think about an **Order** in an e-commerce system. It moves through a clear sequence of states from creation to completion — or cancellation.



Each arrow is a transition triggered by an event — for example, "Customer pays" moves the order from **Created** to **Paid**.

States, Transitions & Guards

STATE DIAGRAM COMPONENTS

1

States

Situations an object can be in: Proposed, Scheduled, Open, Full.

2

Transitions

Arrows triggered by events — e.g., customer pays → move to Paid.

3

Guards

Conditions on transitions — e.g., [seat available] must be true.

4

Actions

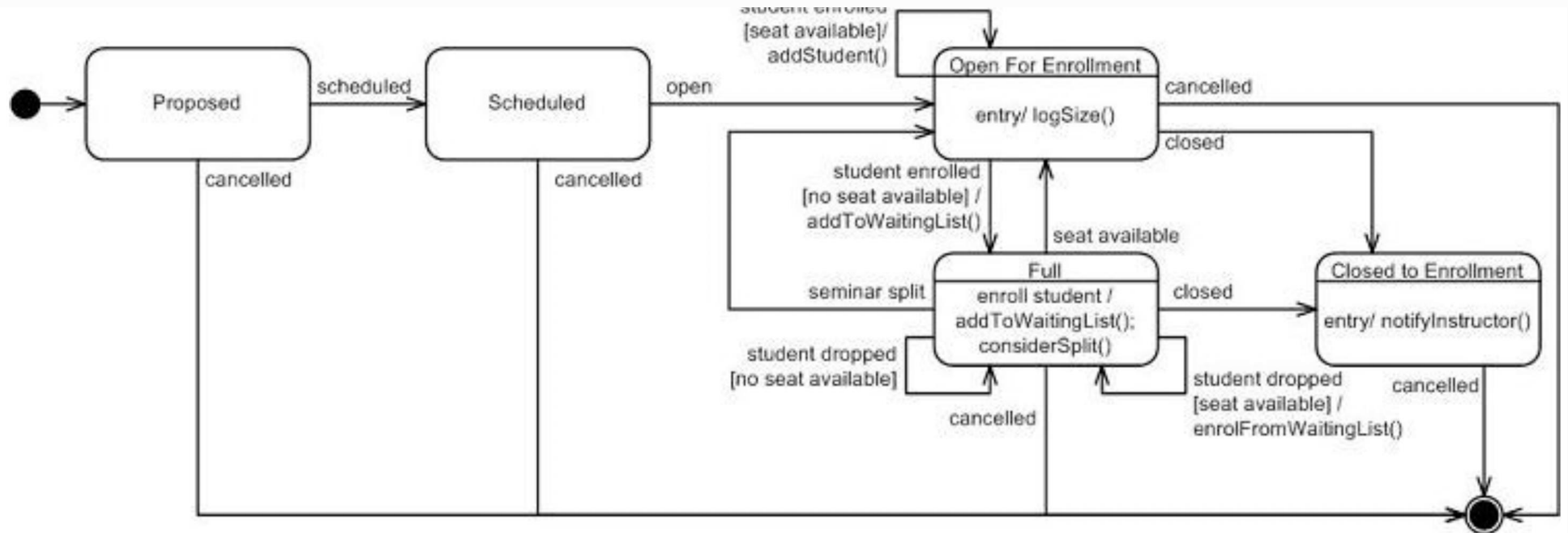
Something the system does during the transition itself.

"State diagram = lifecycle. This is not about communication. This is about evolution."

If you remember one thing about state diagrams, let it be this distinction. Every other UML diagram deals with relationships or structure — the state diagram is uniquely focused on the journey of a single object through time.

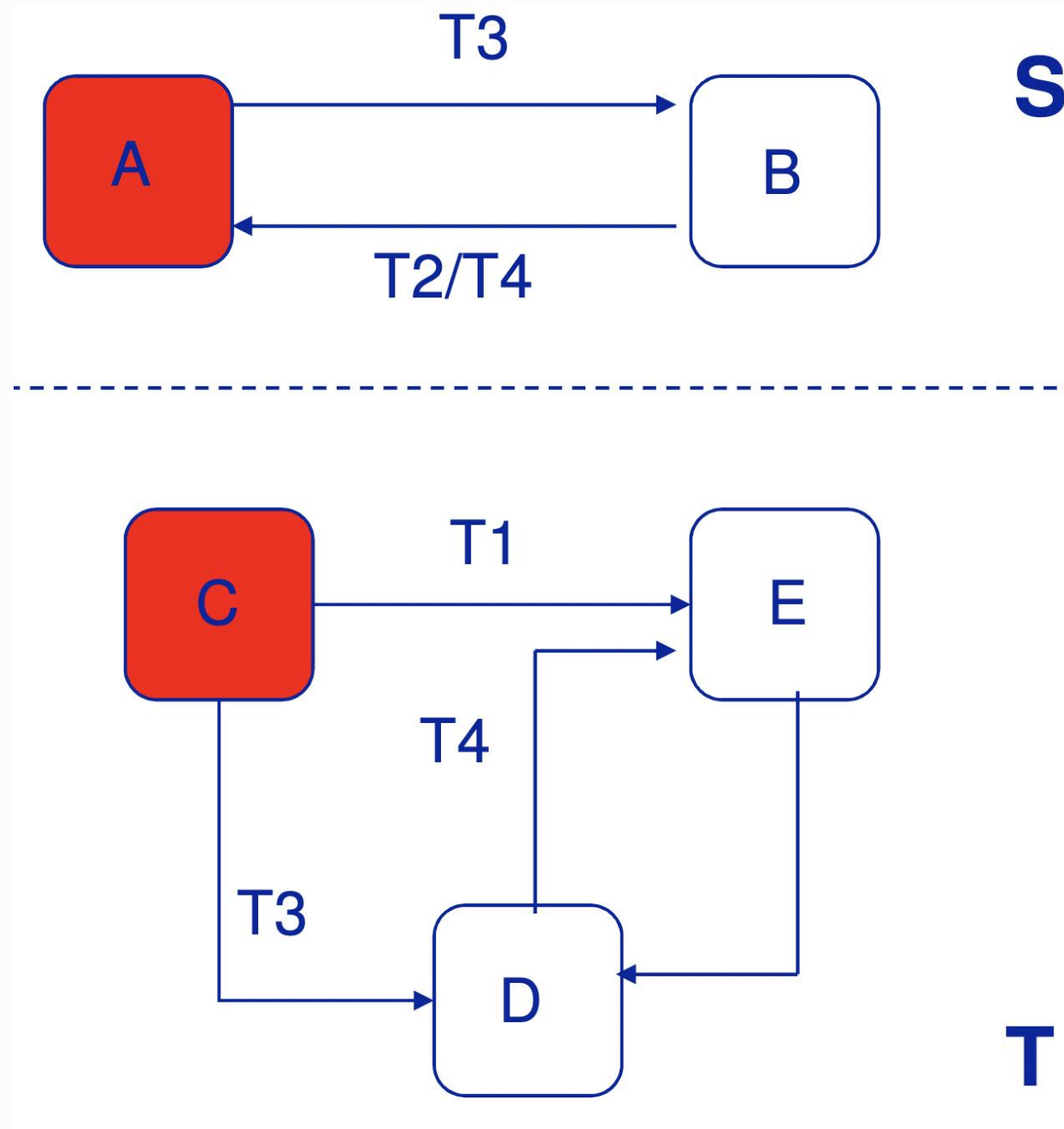
State Diagram: UML Example

UML NOTATION



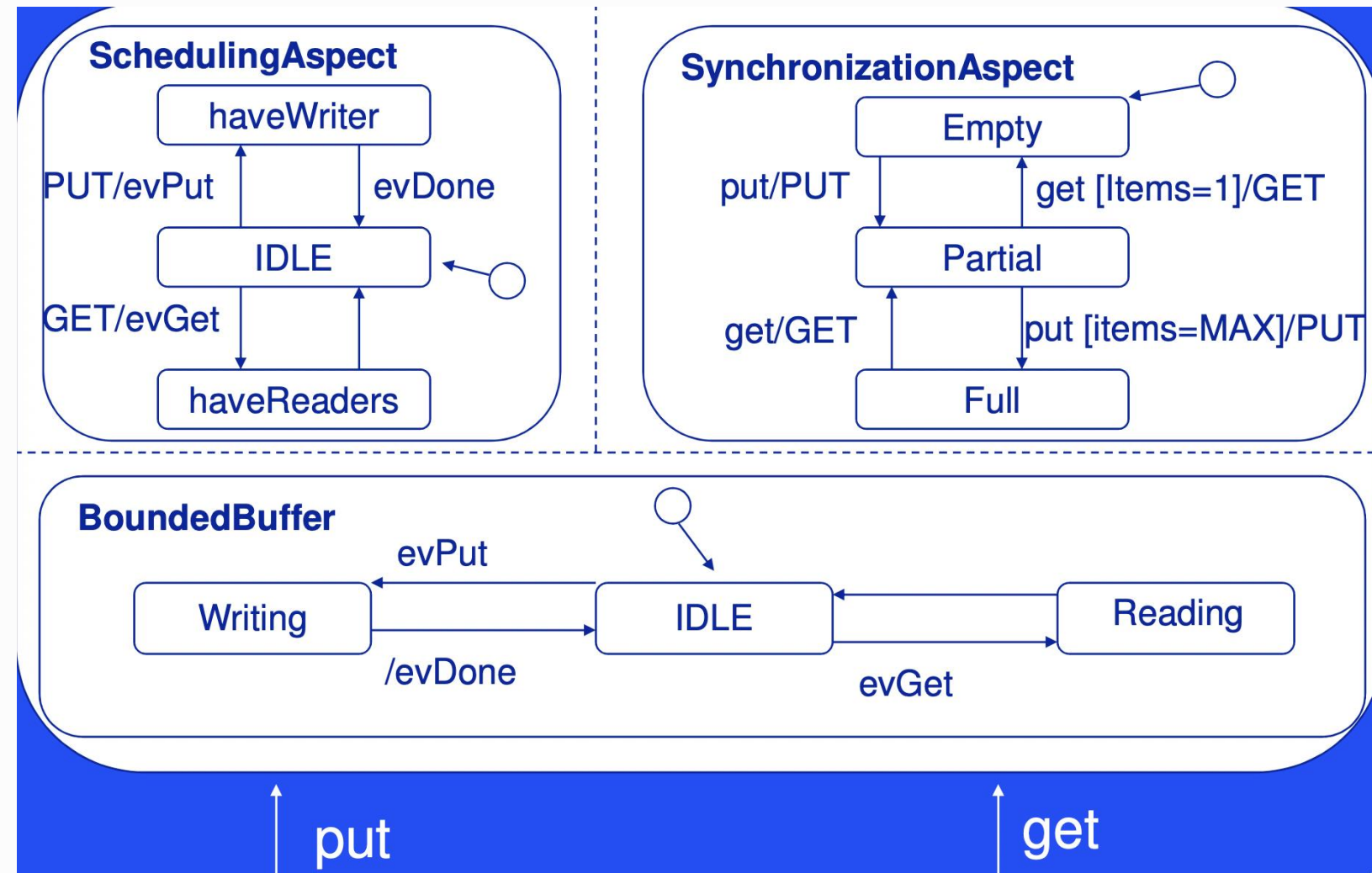
Concurrent State Diagram: UML Example

UML NOTATION



Concurrent State Diagram: The Bounded Buffer

UML NOTATION



Initial state (●) → Pending → Confirmed or Cancelled → Shipped → Delivered → Final state (⊙). Transitions are labeled with the triggering event or method call.

Object Diagrams

SNAPSHOT IN TIME

Class Diagram

A **blueprint**. Describes that a *Customer* and an *Order* exist as types. Abstract. Theoretical.

Object Diagram

A **photograph**. Shows `customer1:Customer` and `order17:Order` — real instances at a specific moment.

Object diagrams show concrete instances with actual values — `Name = John`, `Total = $120`. This is reality, not abstraction.



Class Diagram vs. Object Diagram

Class Diagram = Theory

Defines types, relationships, and structure. The architectural blueprint of your system.

Object Diagram = Reality

Shows what actually exists right now. Concrete instances with real attribute values.

Why Object Diagrams Matter

Object diagrams are a powerful debugging tool for your understanding of a system.

→ Expose Design Errors

If your class diagram is wrong, your object diagram will expose it immediately when you try to instantiate real examples.

→ Validate Relationships

Seeing actual instances with real values confirms whether your class relationships make sense in practice.

→ Communicate Clearly

Stakeholders and developers can more easily understand a concrete snapshot than an abstract blueprint.

Object Diagram: Example

CONCRETE INSTANCES

Instead of abstract classes, we show real objects with real data. Here is what a simple e-commerce snapshot looks like:

customer1 : Customer

name = "John" email =
"john@email.com"

order17 : Order

total = \$120 status = "Paid"

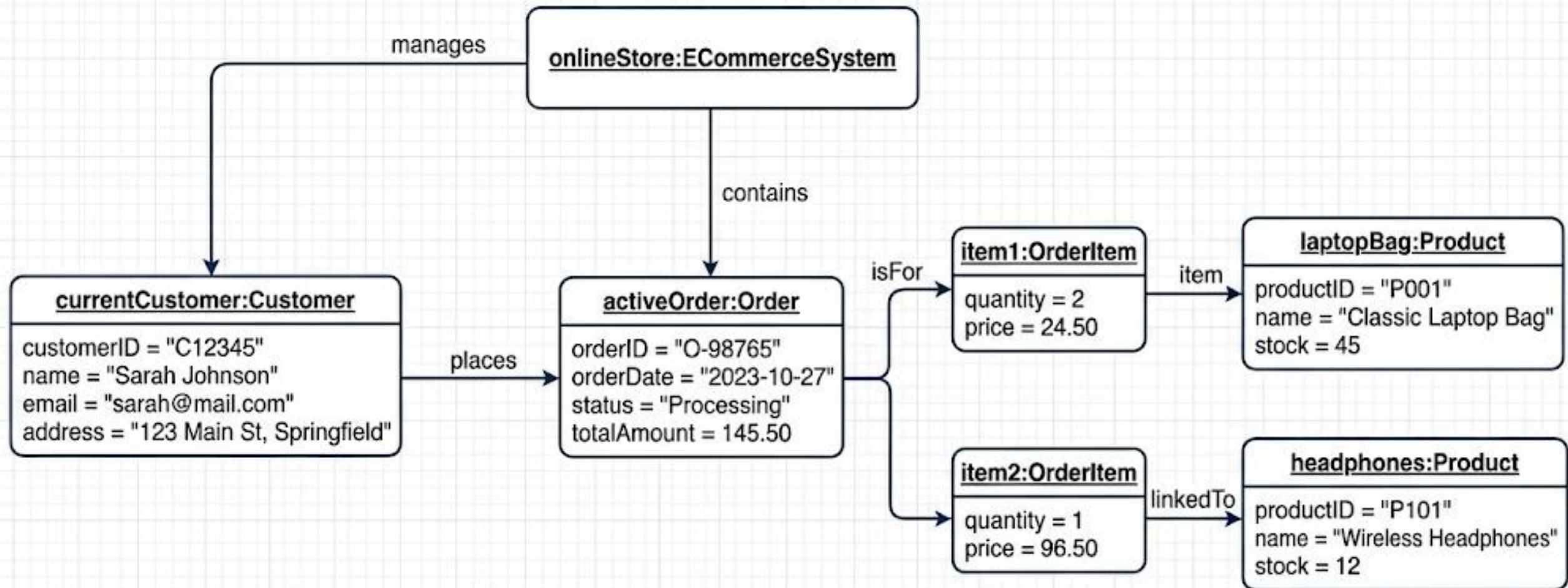
product5 : Product

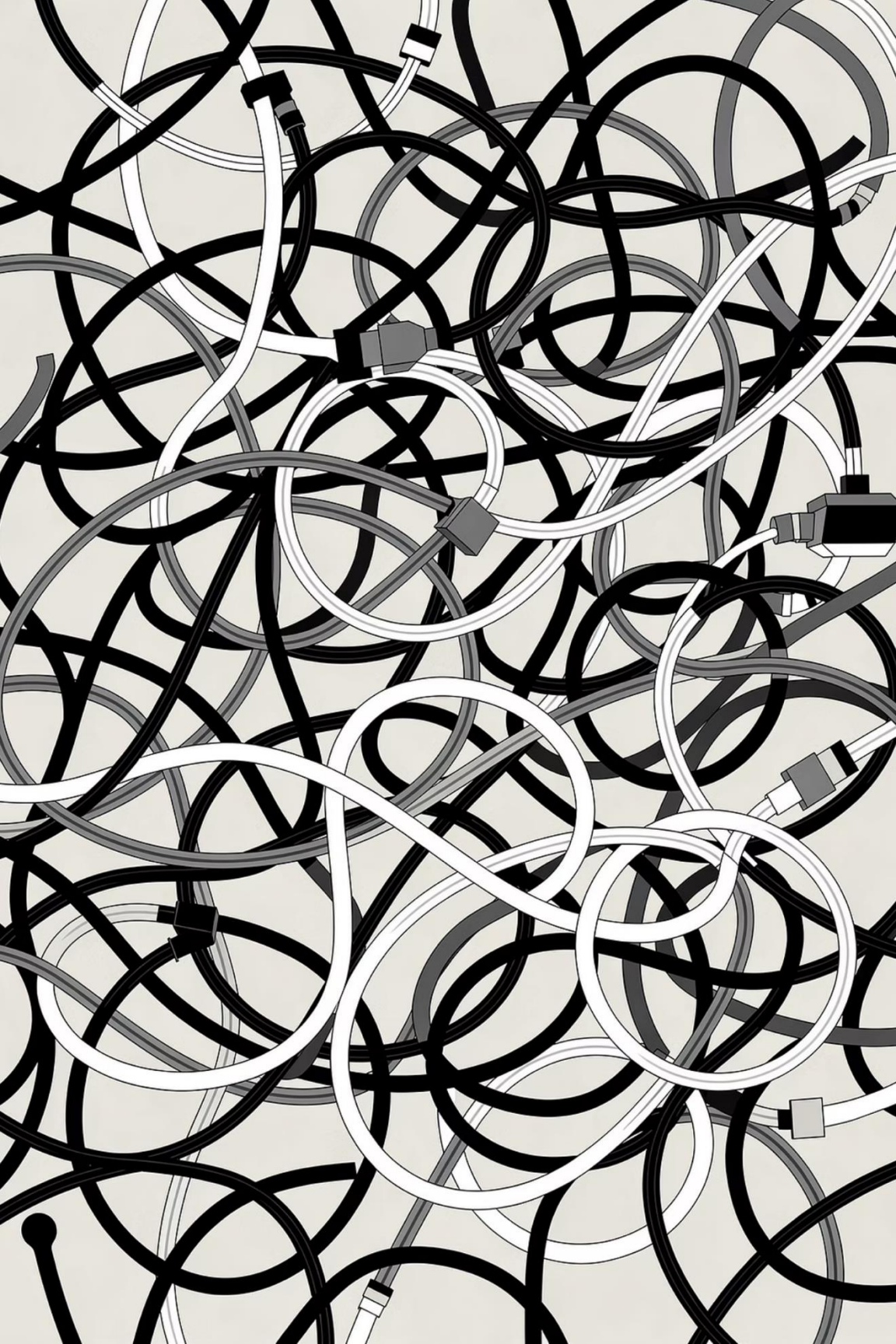
name = "Laptop" price = \$120

Each object is an instance of a class, linked by real associations — a photograph of the system at one moment in time.

Object Diagram: UML Example

UML NOTATION





The Problem with Growing Systems

As systems grow, class diagrams become unmanageable. Hundreds of classes, thousands of relationships — the diagram turns into **spaghetti**. You can no longer see the architecture. You can no longer answer the most basic question: *How is this system organized?*



This is where Package Diagrams save your life.

Package Diagrams

ORGANIZATION

A **package** is simply a group of classes — think of it like a folder. Package diagrams let you see the high-level architecture of a system without drowning in detail.



UI

Presentation layer — what the user sees and interacts with.



Application

Orchestrates use cases and business workflows.



Domain

Core business logic and entities.

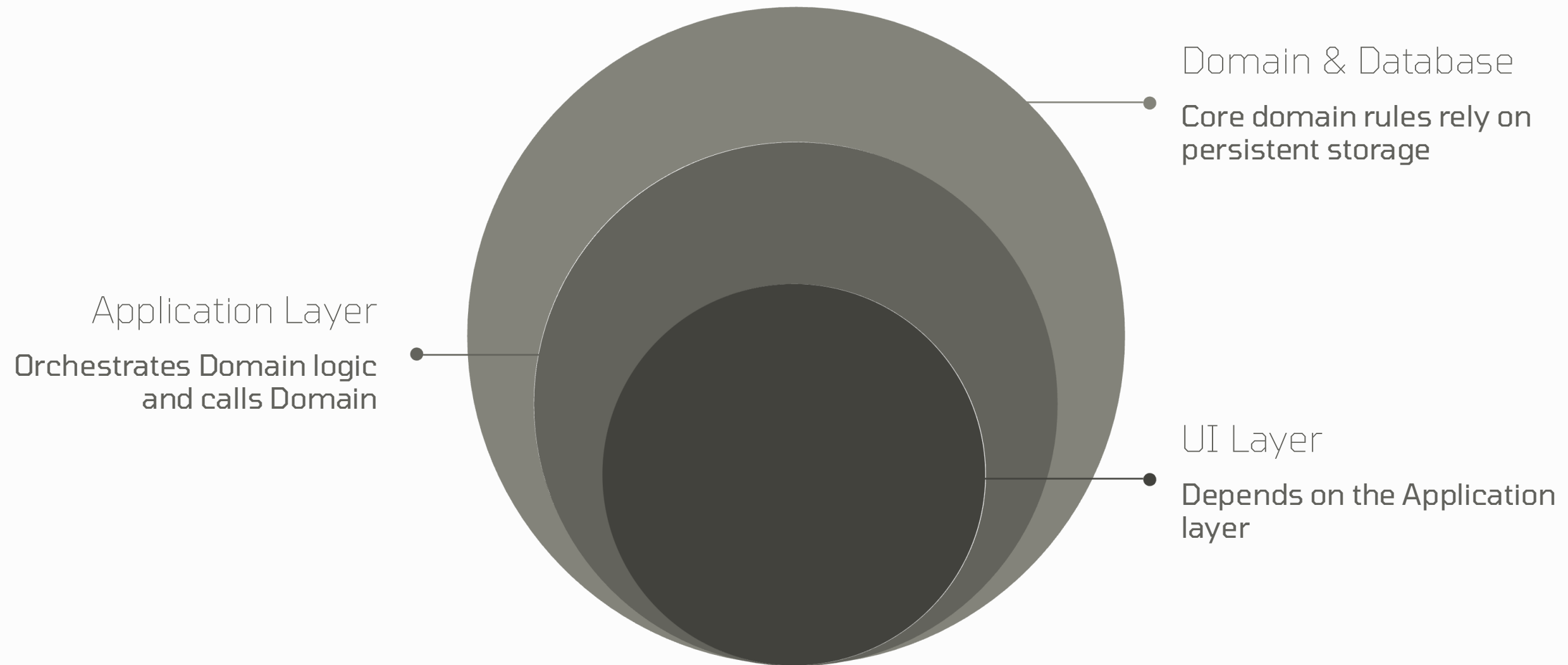


Database

Persistence and data access layer.

Dependencies Are Everything

In a package diagram, the boxes matter less than the **arrows between them**. Dependencies tell you who relies on whom — and that defines your architecture.



This one-directional dependency chain is the foundation of clean, maintainable architecture. Violations of this flow create tight coupling and fragile systems.

Why Package Diagrams Matter

PRACTICAL ARCHITECTURE

The Key Question

Package diagrams answer: **How is the system organized?**

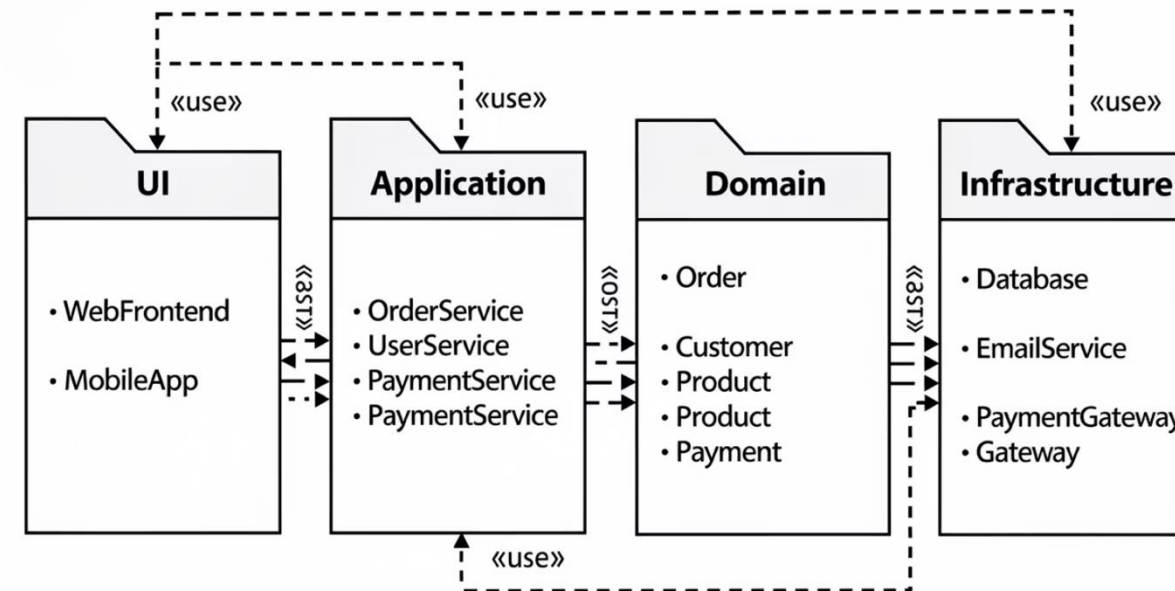
If you're designing a real system, this is one of the most important diagrams you will draw. It gives every developer a shared map of the codebase.

What You Can See

- High-level architecture at a glance
- Which layers depend on which
- Where coupling risks exist
- How to split teams by module

Package Diagram: UML Example

UML NOTATION

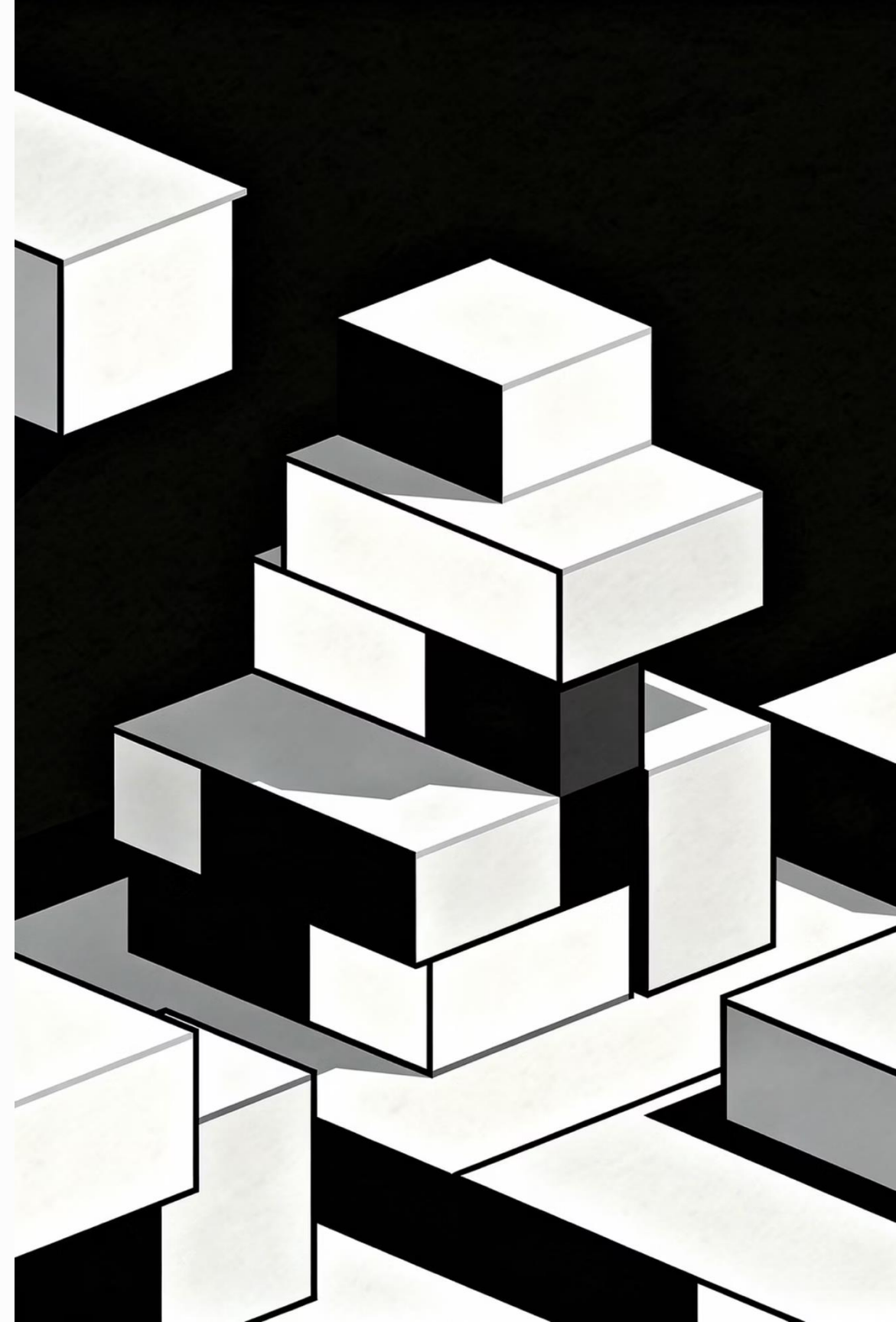


Packages are drawn as folder tabs. Dashed «use» arrows show dependencies flowing downward: UI → Application → Domain and Infrastructure. The direction of arrows = direction of coupling.

From Logical to Physical

Package diagrams show logical groupings. But when we ask *how is the system actually built* — we need to go one level deeper. We move from folders of classes to deployable, physical building blocks.

This is the domain of **Component Diagrams**.



Component Diagrams

BUILD STRUCTURE

Component diagrams show the **actual parts of the system** — things you can deploy. A component is a service, a module, or a microservice. This is very close to real architecture.

Frontend

The user-facing application — web or mobile client.

API

The gateway that routes requests to internal services.

Auth Service

Handles authentication and authorization.

Payment Service

Processes transactions and integrates with payment providers.

Interfaces: Provided & Required

Component diagrams also show **interfaces** — what each component provides to others, and what it requires from others. This makes the contract between components explicit.

Provided Interface

What this component exposes — e.g., the API provides a REST `/orders` endpoint.

Required Interface

What this component needs — e.g., the API requires the Auth Service to validate tokens.

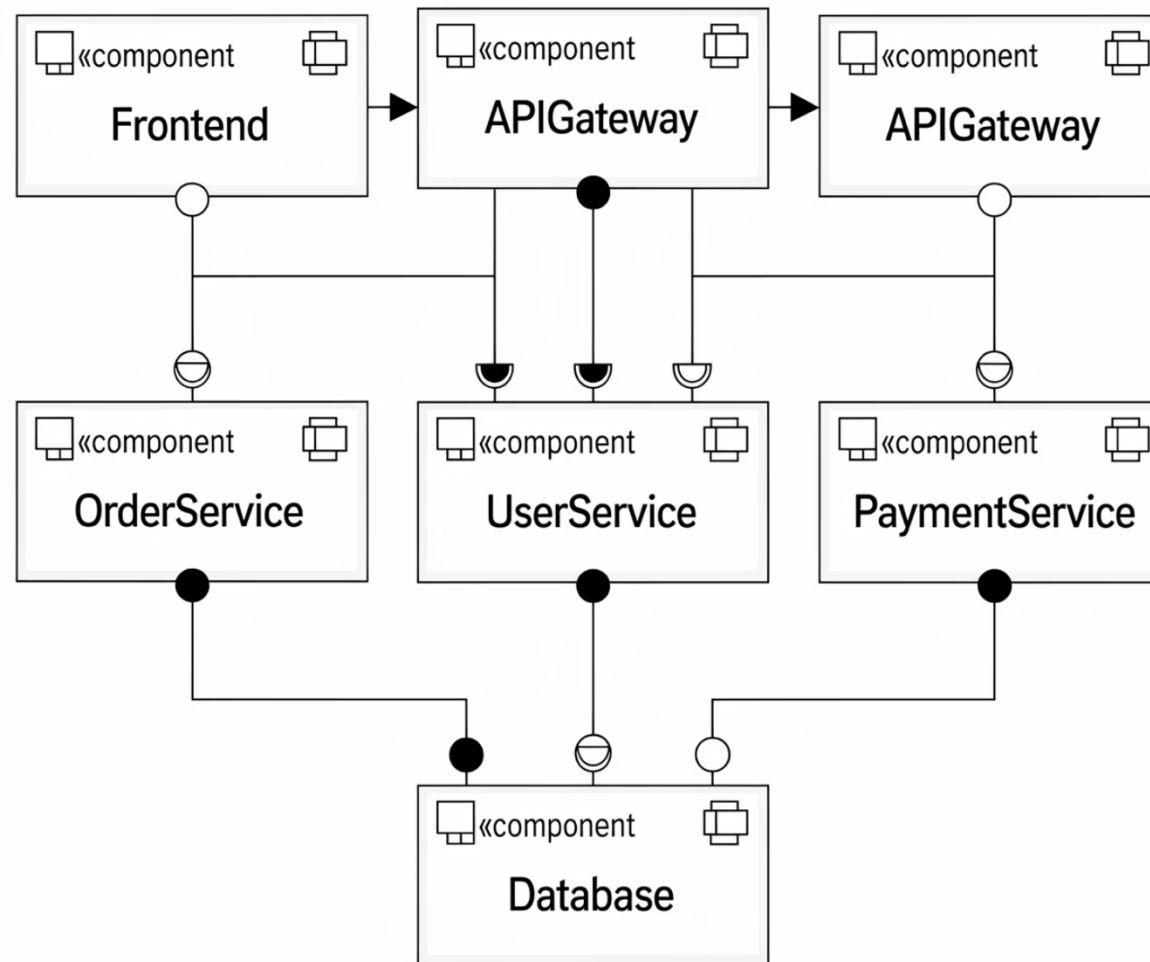
 In modern systems, components are your microservices. Component diagrams map your microservice architecture.

"A component is something you can deploy. A service, a module, a microservice."

This distinction is critical. Component diagrams bridge the gap between logical design and physical deployment — they show the real building blocks that engineers actually ship to production.

Component Diagram: UML Example

UML NOTATION



Components are deployable units shown with the «component» stereotype. Lollipop (●—) = provided interface. Socket (—○) = required interface. Connections show which components depend on which.

Deployment Diagrams

WHERE IT RUNS

Deployment diagrams show the **physical world**. Where does everything actually run? This is where Dev meets Ops — the bridge between software architecture and infrastructure.



Nodes & Artifacts

Deployment diagrams are built from two core concepts:

Nodes

Physical or virtual execution environments — servers, machines, containers, cloud instances. These are the *where*.

Artifacts

The deployable software units that run on nodes — executables, services, databases. These are the *what*.

The diagram shows which artifacts are deployed on which nodes, and how those nodes communicate with each other.

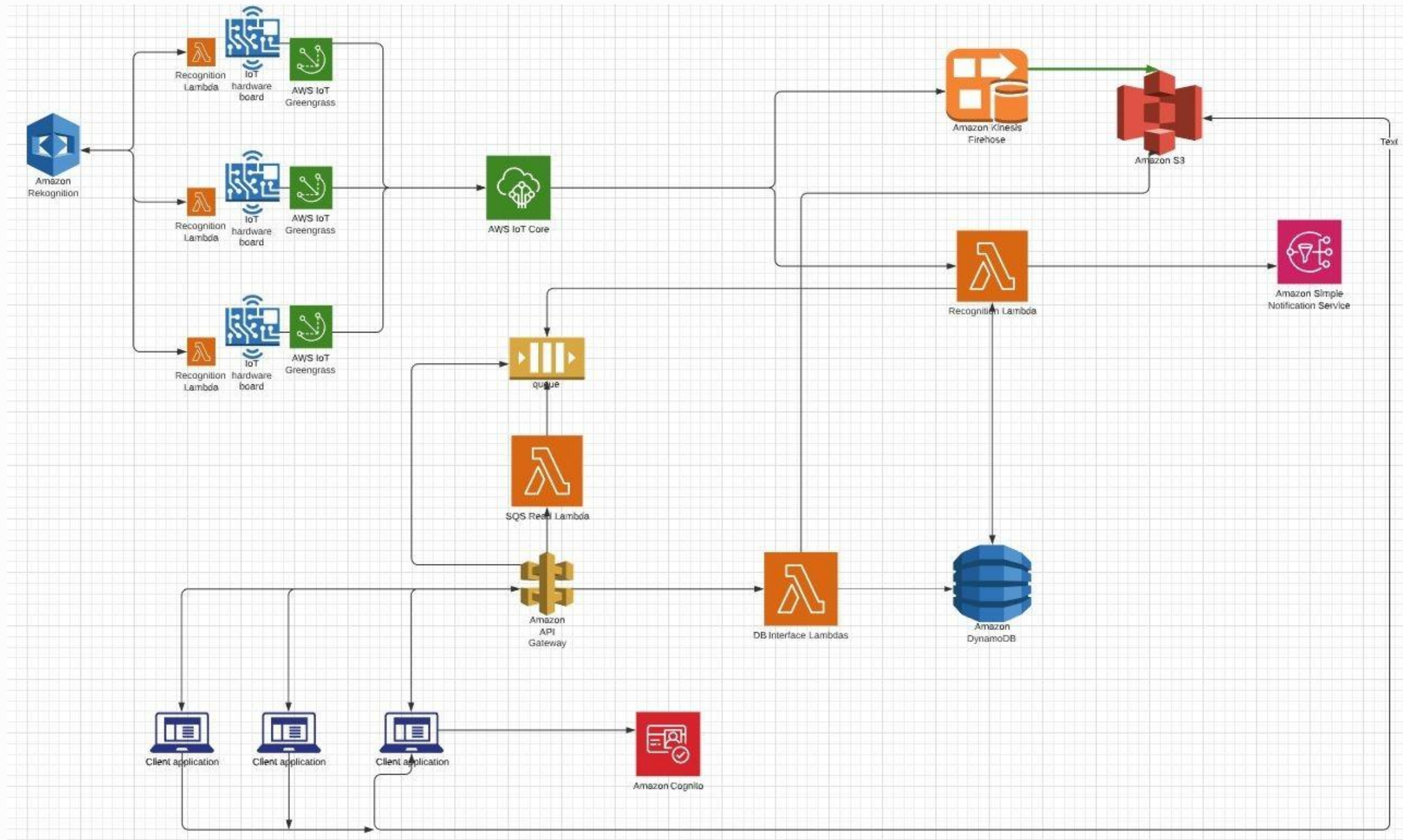
Deployment Diagram: Classic Web Stack



This classic architecture maps directly to the physical machines in a data center. Each node is a server, and the arrows represent network communication between them.

Deployment Diagram: UML Example

UML NOTATION



Modern Deployment: Cloud & Kubernetes

DEV MEETS OPS

In modern systems, deployment diagrams evolve to reflect cloud-native infrastructure.



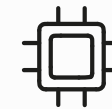
Cloud Nodes

AWS, Azure, GCP — virtual machines and managed services replace physical servers.



Kubernetes

Containers orchestrated across clusters — pods and services replace traditional server deployments.



Specialized Hardware

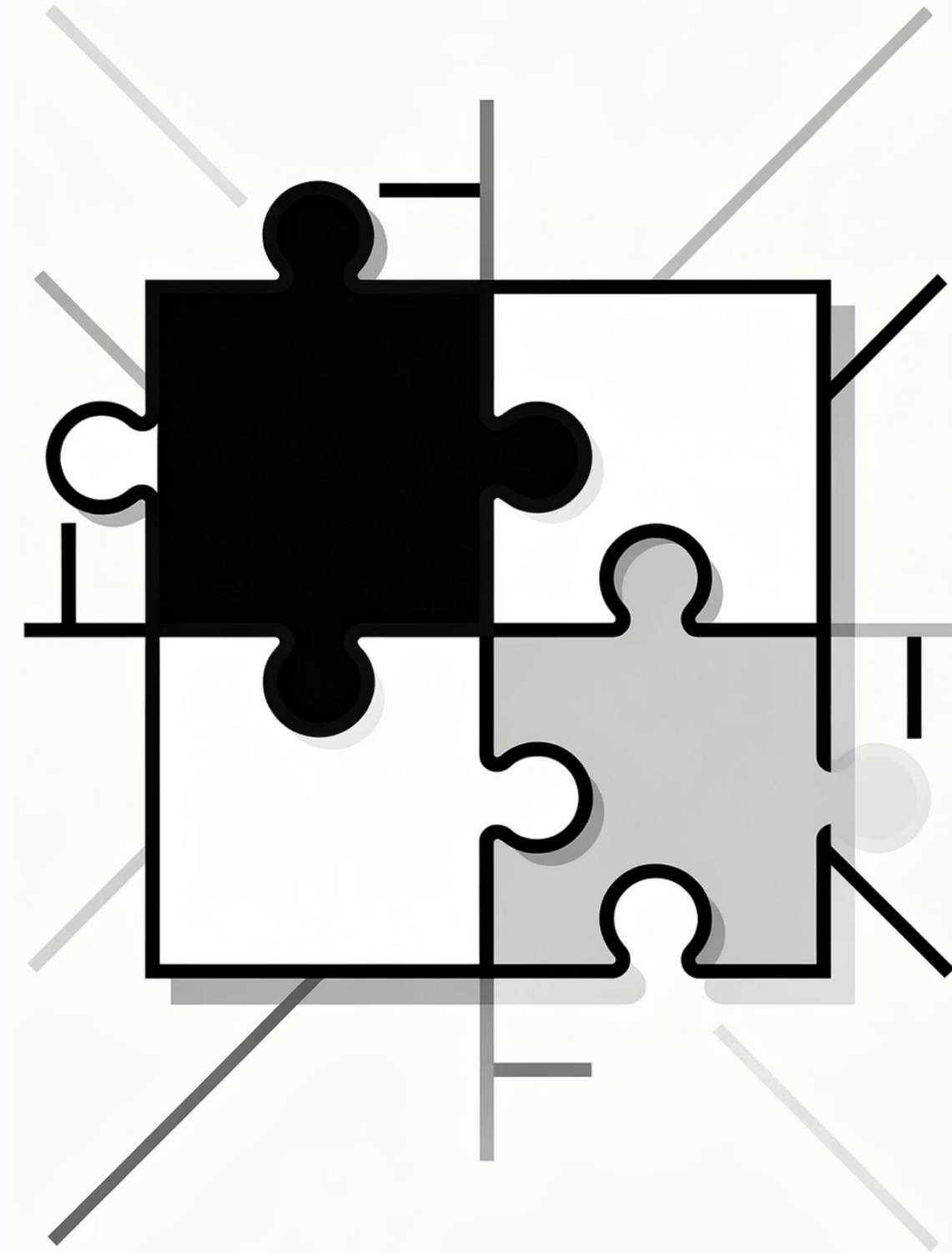
GPUs and TPUs for AI workloads — deployment diagrams capture these specialized nodes too.

"This is where Dev meets Ops."

Deployment diagrams are the language shared between software engineers and infrastructure teams. They answer the single most operational question in any system: **Where does it run?**

The Five Diagrams Together

Each diagram answers a different question. Together, they give you a complete picture of any software system — from the lifecycle of a single object to the physical machines running in production.



Complete UML Summary

1

State Diagram

How an object changes over time — lifecycle and evolution.

2

Object Diagram

What exists right now — a snapshot of concrete instances.

3

Package Diagram

How the system is organized — logical groupings and dependencies.

4

Component Diagram

How it is built — deployable parts and their interfaces.

5

Deployment Diagram

Where it runs — physical nodes and infrastructure.

Diagram Comparison at a Glance

Diagram	Focus	Key Question	Example
State	Single object lifecycle	How does it evolve?	Order: Created → Paid → Shipped
Object	Concrete instances	What exists now?	customer1:Customer, name="John"
Package	Logical grouping	How is it organized?	UI → Application → Domain
Component	Deployable parts	How is it built?	Frontend, API, Auth Service
Deployment	Physical infrastructure	Where does it run?	Browser → Web Server → DB

State Diagram: Quick Recap

REVIEW

Remember

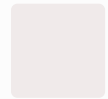
- Describes **ONE** object's lifecycle
- States = situations the object can be in
- Transitions = triggered by events
- Guards = conditions like `[seat available]`
- Actions = behavior during transition

Common Mistake

Students often confuse state diagrams with sequence diagrams.
Remember: state diagrams are about **one object evolving**, not about objects communicating with each other.

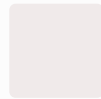
Object Diagram: Quick Recap

REVIEW



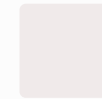
It's a photograph, not a blueprint

Shows real instances at a specific moment — not abstract types.



Includes actual attribute values

Not just `Customer` — but `customer1:Customer` with `name="John"`.



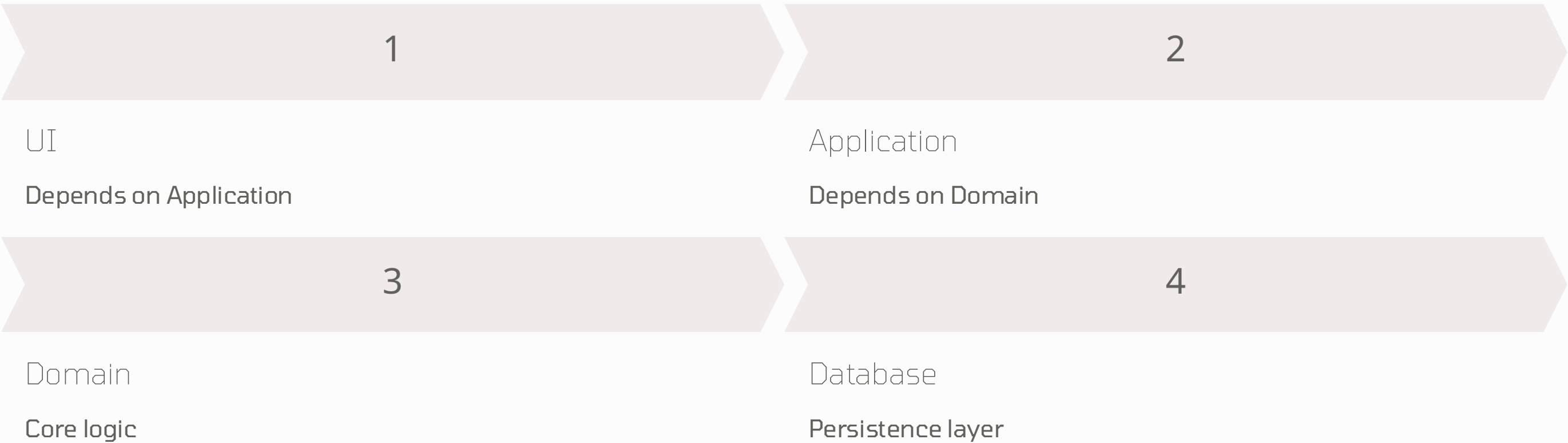
Exposes class diagram errors

If your class diagram is wrong, the object diagram will reveal it immediately.

Package Diagram: Quick Recap

REVIEW

Packages are folders for classes. The diagram shows high-level architecture and — most importantly — **dependencies between packages**.



This layered dependency structure is the foundation of clean, scalable system design.

Component Diagram: Quick Recap

REVIEW

What It Shows

- Deployable parts of the system
- Provided and required interfaces
- Dependencies between components

Modern Equivalent

In today's systems, components are **microservices**. The component diagram is your microservice architecture map.

Deployment Diagram: Quick Recap

REVIEW



Nodes

Servers, machines, containers — the physical or virtual environments where software runs.



Artifacts

The software deployed on each node — services, executables, databases.



Communication

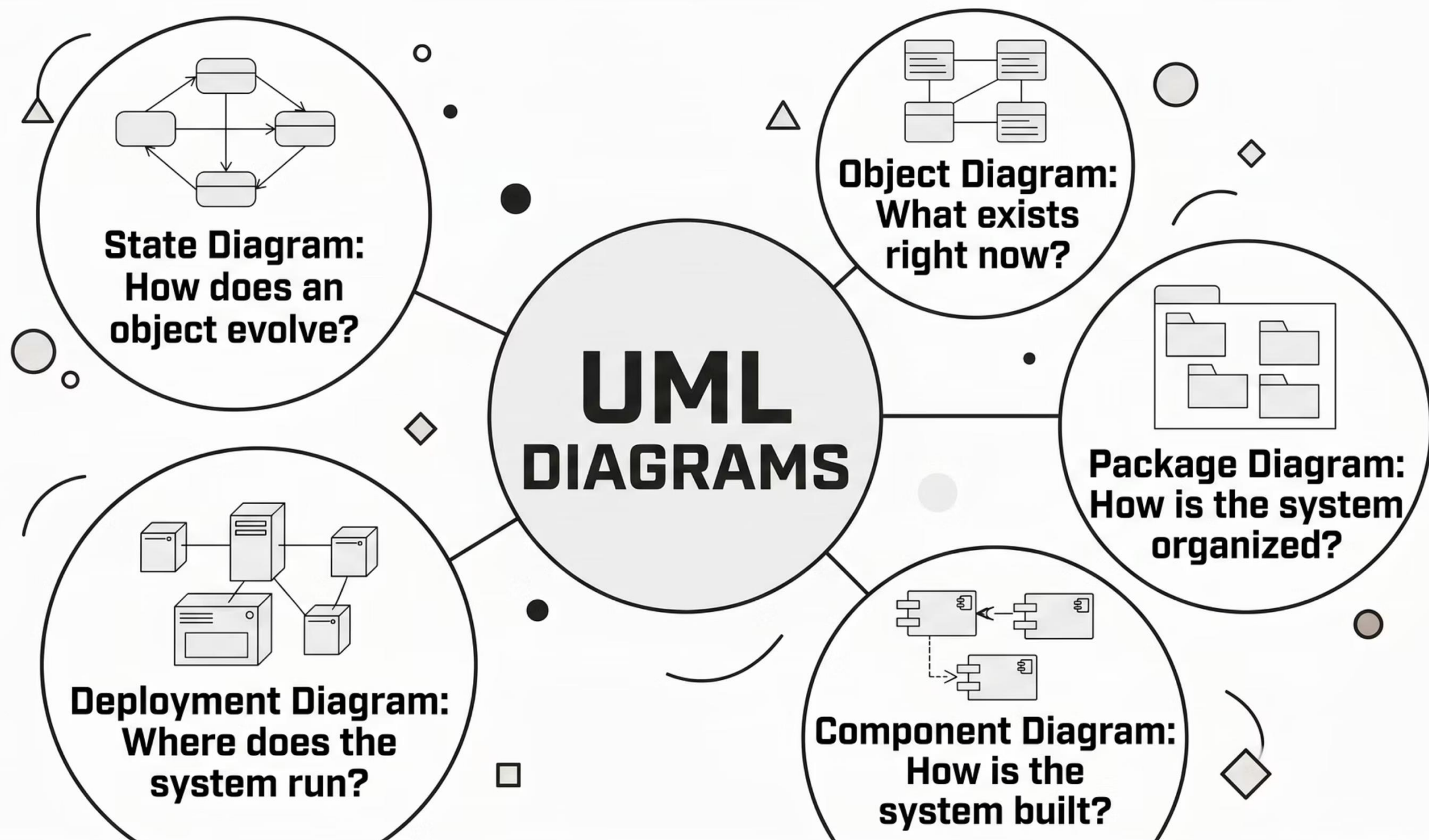
Arrows show how nodes communicate — HTTP, TCP, message queues.



From Idea to Production

These five diagrams together form a complete toolkit. A software architect who masters all five can describe any system at any level of abstraction — from the behavior of a single object to the physical infrastructure running in the cloud.

Which Diagram for Which Question?



Practical Application

When you encounter a real system, use these diagrams as lenses:



Start with Deployment

Understand where the system runs — nodes, servers, cloud.



Then Package

See how the codebase is logically organized and layered.



Then Component

Identify the deployable building blocks and their interfaces.



Then State & Object

Understand how key entities evolve and what they look like at runtime.

Common Exam Pitfalls

WATCH OUT

State $\neq$ Sequence

State diagrams show one object evolving. Sequence diagrams show multiple objects communicating. Don't confuse them.

Object $\neq$ Class

An object diagram shows instances with real values, not abstract types. Always use the `instanceName:ClassName` notation.

Component $\neq$ Package

Packages are logical groupings of code. Components are physical, deployable units. The distinction is logical vs. physical.

Key Vocabulary

Term	Definition
State	A situation an object can be in at a given point in time
Transition	A change from one state to another, triggered by an event
Guard	A condition that must be true for a transition to occur
Package	A named group of related classes — like a folder
Component	A deployable, replaceable part of the system
Node	A physical or virtual execution environment in a deployment diagram
Artifact	A deployable software unit placed on a node



QUESTIONS???